

Prompt Engineering

Claude & GPT-5 Codex Models

Model-specific techniques for getting the best results

What We'll Cover

Claude (Anthropic)

- Communication style & verbosity
- Model self-knowledge & API strings

GPT-5 Codex (OpenAI)

- Autonomy & persistence prompting
- Code implementation guidelines
- Codebase exploration patterns
- AGENTS.md convention
- Plan tool hygiene
- Preambles, personality & metaprompting

Claude

Anthropic Claude Opus 4 · Sonnet 4 · Haiku 3.5

Claude – Communication Style

Claude's latest models are more concise and natural than earlier versions.

Behaviour	Description
More direct	Fact-based progress reports, not self-celebratory updates
More conversational	Slightly fluent and colloquial, less machine-like
Less verbose	May skip summaries after tool calls for efficiency

If you want more visibility into its reasoning:

```
After completing a task that involves tool use, provide a quick
summary of the work you've done.
```

Claude – Model Self-Knowledge

Set model identity explicitly in your system prompt:

```
The assistant is Claude, created by Anthropic.  
The current model is Claude Sonnet 4.5.
```

For apps that need specific API model strings:

```
When an LLM is needed, default to Claude Sonnet 4.5 unless the  
user requests otherwise.  
The exact model string is: claude-sonnet-4-5
```

Claude – Current Model Strings

Model	API string
Claude Opus 4	<code>claude-opus-4-5</code>
Claude Sonnet 4	<code>claude-sonnet-4-5</code>
Claude Haiku 3.5	<code>claude-haiku-3-5</code>

Always check the [Anthropic models overview](#) for the latest strings – they are updated frequently.

GPT-5 Codex

OpenAI `gpt-5.X-codex` Agentic Coding Models

Codex – Model Overview

Codex is OpenAI's recommended **agentic coding model**.

- Acts as an **autonomous senior engineer** – gathers context, plans, implements, tests, and refines without prompting at each step
- Supports **reasoning effort levels**: `medium` (recommended), `high`, `xhigh`
- Has **first-class compaction support** for multi-hour sessions

Codex – Autonomy & Persistence

Unlike general models, Codex is designed to persist until fully done.

You are an autonomous senior engineer. Once given a direction, proactively gather context, plan, implement, test, and refine without waiting for additional prompts. Persist until the task is fully handled end-to-end within the current turn. Only end your turn when the problem is solved. Bias to implementing with reasonable assumptions; do not end on clarifications unless truly blocked.

Important: Do NOT prompt for upfront plans or status updates during a rollout – this can cause the model to stop early.

Codex – Code Implementation Guidelines

Principle	What to include in your prompt
Root cause focus	Fix the underlying issue, not just symptoms
Codebase conventions	Follow existing patterns, naming, and formatting
Tight error handling	No broad <code>try/catch</code> ; surface errors explicitly
Type safety	Avoid <code>as any</code> ; prefer proper types and guards
DRY / search first	Search for prior art before adding new helpers
Batch edits	Read enough context before editing; avoid micro-edits
Completeness	Wire all relevant surfaces; keep behaviour consistent

Codex – Codebase Exploration

Tell the model how to navigate efficiently:

```
When searching for text or files, prefer using `rg` or `rg --files`  
because `rg` is much faster than alternatives like `grep`.  
Before making tool calls, decide all files you will need and read  
them together in a single parallel batch.
```

Key rules:

- Use `rg` (ripgrep) over `grep`
- **Think first** – decide all files needed before any tool call
- **Batch reads** – read multiple files in one parallel call
- Only go sequential when you genuinely can't know the next file without seeing a result first

Codex – AGENTS.md

Codex supports per-directory instruction files called `AGENTS.md` .

- Place in `~/.codex` or any directory from repo root to cwd
- Files are **merged in order** – deeper directories override earlier ones
- Each file is injected as its own user-role message near the top of history

Format: `# AGENTS.md instructions for <directory>`

Use for: project coding standards, naming conventions, tooling preferences – without repeating them in every prompt.

Codex – Plan Tool Hygiene

- **Skip the plan tool** for straightforward tasks (~easiest 25%)
- Do not make single-step plans
- Update the plan after completing each sub-task
- **Never end with only a plan** – the deliverable is working code
- Before finishing, reconcile every TODO as `Done` , `Blocked` , or `Cancelled`
- Do not commit to tests/refactors unless doing them immediately

Codex – Personality Modes

Friendly

Warm, partner-like pairing energy. Better for onboarding, ambiguous tasks, or higher-stakes changes.

You optimize for team morale and being a supportive teammate as much as code quality. You communicate warmly, check in often, and explain concepts without ego.

Pragmatic

Terse, direct, higher ratio of actionable information per token. Better when latency matters.

- Fewer social flourishes
- Focus on progress and results

Codex – Preamble Best Practices

Preambles are short progress messages sent alongside tool calls.

- **1 sentence** acknowledgement + **1-2 sentence** plan before any tool calls
- Most updates: **1-2 sentences**; longer only at real milestones
- **Cadence:** at least every 1-3 execution steps; hard floor every 6 steps or 10 tool calls
- **Cover:** outcome so far · next 1-3 steps · open questions
- **Tone:** real person pairing – no headings, status labels, or log-style language

Codex – Metaprompting

When Codex produces a good result but is slow or exhibits bad patterns, ask it to improve its own instructions:

```
That was a high quality response, thanks! It seemed like it took  
a while though. Is there a way to clarify your instructions so you  
can get to a response as good as this faster next time?
```

```
Read through your instructions and look for anything that made you  
take longer than needed. Write targeted (but generalised)  
additions/changes/deletions to make a request like this faster  
next time with the same quality.
```

Common failure modes to fix: slow starts · loggy status updates · repetitive tics like "Good catch", "Aha", "Got it-"

Codex – Editing Constraints

Safe, predictable file editing guardrails:

- Default to **ASCII**; only introduce Unicode when justified
- Add comments only where code is not self-explanatory
- **Never revert changes you did not make** unless asked
- Do **not** amend commits unless explicitly asked
- Never use destructive git commands (`git reset --hard` , `git checkout --`) without approval
- If unexpected changes are noticed, **stop and ask** the user how to proceed

Summary – Claude vs Codex

Topic	Claude	GPT-5 Codex
Style	Concise, conversational	Autonomous, persistence-focused
Identity	Set via system prompt	N/A
Task approach	Responds per turn	Runs end-to-end autonomously
Codebase nav	Standard	<code>rg</code> , parallel batch reads
Per-project rules	System prompt	<code>AGENTS.md</code> files
Self-improvement	Adjust prompt	Metaprompting
Tone control	Role instructions	Friendly / Pragmatic modes

Happy Prompting!

Apply these model-specific techniques alongside the general prompt engineering best practices.